



TECHNISCH ONTWERP

Theremin

SAMENVATTING

Dit document bevat het technisch ontwerp van het theremin project

Sem Overbeek

ESE Leertaak 2

[Vul hier de AIAS in]

Versiebeheer

Versie	Naam	Wijziging	Datum
0.1	Niels Tielenburg	eerste conceptversie	01-05-25
0.2	Niels Tielenburg		13-05-25
0.9	Willie Conen	Invulling met C4	04-05-25
1.0	bRAM	Verwijzingen naar FO toegevoegd, filter sortering aangepast met plaatjes. Kleine tekstuele wijzigingen	10-06-2025
1.1	Niels Tielenburg	AIAS toegevoegd	08-09-25
1.2	Niels Tielenburg	C3 niveau plaatje aangepast	23-09-25
2.0	Sem Overbeek	TO afgemaakt	10-10-2025

Inhoudsopgave

Versiebeheer	1
Inhoudsopgave	3
Inleiding	5
1 Context	6
2 Containers	6
3 Componenten	6
3.1 Componenten	8
3.2 Hardware overzicht	8
4 Controller	9
4.1 Main	9
5 Volume regelaar	10
5.1 ADMUX instellingen	10
5.2 ADCSRA instellingen	11
5.3 ADCSRB instellingen	11
6 Frequency	11
6.1 Tijdmeting	12
6.1.1 TCCR1A instellingen	13
6.1.2 TCCR1B instellingen	13
6.1.3 TIMSK1 instellingen	14
7 Filter	15
7.1 Quicksort	16
8 Filter Instellen	16
8.1 PCICR en PCMSK2	16
9 Buzzer interface	17
9.1 Volume (fast PWM)	18
9.1.1 TCCR2A instellingen	18
9.1.2 TCCR2B instellingen	18
9.1.3 TIMSK2 instellingen	18
9.1.4 OCR2B instellingen	18
9.2 Frequentie	18
9.2.1 TCCR0A instellingen	19
9.2.2 TCCR0B instellingen	19
9.2.3 TIMSK2 instellingen	20
10 LCD scherm	21

11	7-Segmenten display	21
----	---------------------------	----

Inleiding

Een dirigent heeft een briljant idee om een orkest samen te stellen met muzikanten die op een theremin kunnen spelen. Het uiteindelijke doel is om een muziekstuk in te studeren en een concert te geven. Omdat theremins niet zo gangbaar zijn als muziekinstrument wil de dirigent dat er een prototype gemaakt wordt van een zelf te ontwikkelen muziekinstrument die later in productie kan worden genomen.

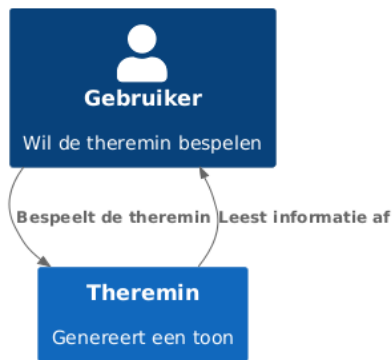
Een Theremin is een elektronisch muziekinstrument. De bespeler kan met één hand de hoogte van de toon variëren (rechttopstaande antenne) en met de andere het volume (de loop antenne). Zie voor de werking o.a.

<https://en.wikipedia.org/wiki/Theremin>.

In (ESA-team, 2025) staat het functionele ontwerp van de theremin beschreven. De oplossing voor de gevraagde functionaliteiten zijn in dit document technisch uitgewerkt. Verwijzing naar de betreffende use case of requirement is aangegeven met US.1, RV.1, etc. Het technisch ontwerp wordt aan de hand van het C4 model uitgediept. Per laag wordt in een apart hoofdstuk beschreven hoe het systeem eruit moet komen te zien.

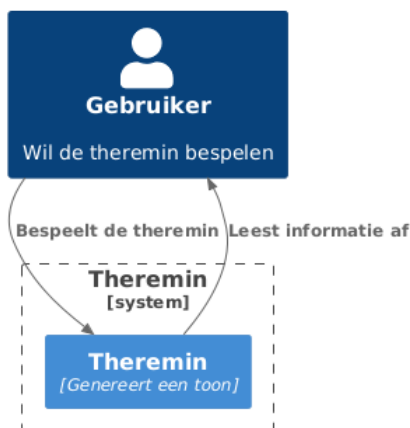
1 Context

Het systeem dat wordt ontwikkeld gaat gebruikt worden als een muziekinstrument. De muzikant bespeelt de theremin door met de hand de toonhoogte te beïnvloeden, en met de andere hand de volume te beïnvloeden. Daarnaast kan de gebruiker informatie uitlezen over de actuele status, zoals welke toonhoogte er op dit moment wordt gegenereerd, wat de afstand is van de theremin tot aan de hand van de gebruiker en wat de huidige grootte is van het filter dat wordt gebruikt.



2 Containers

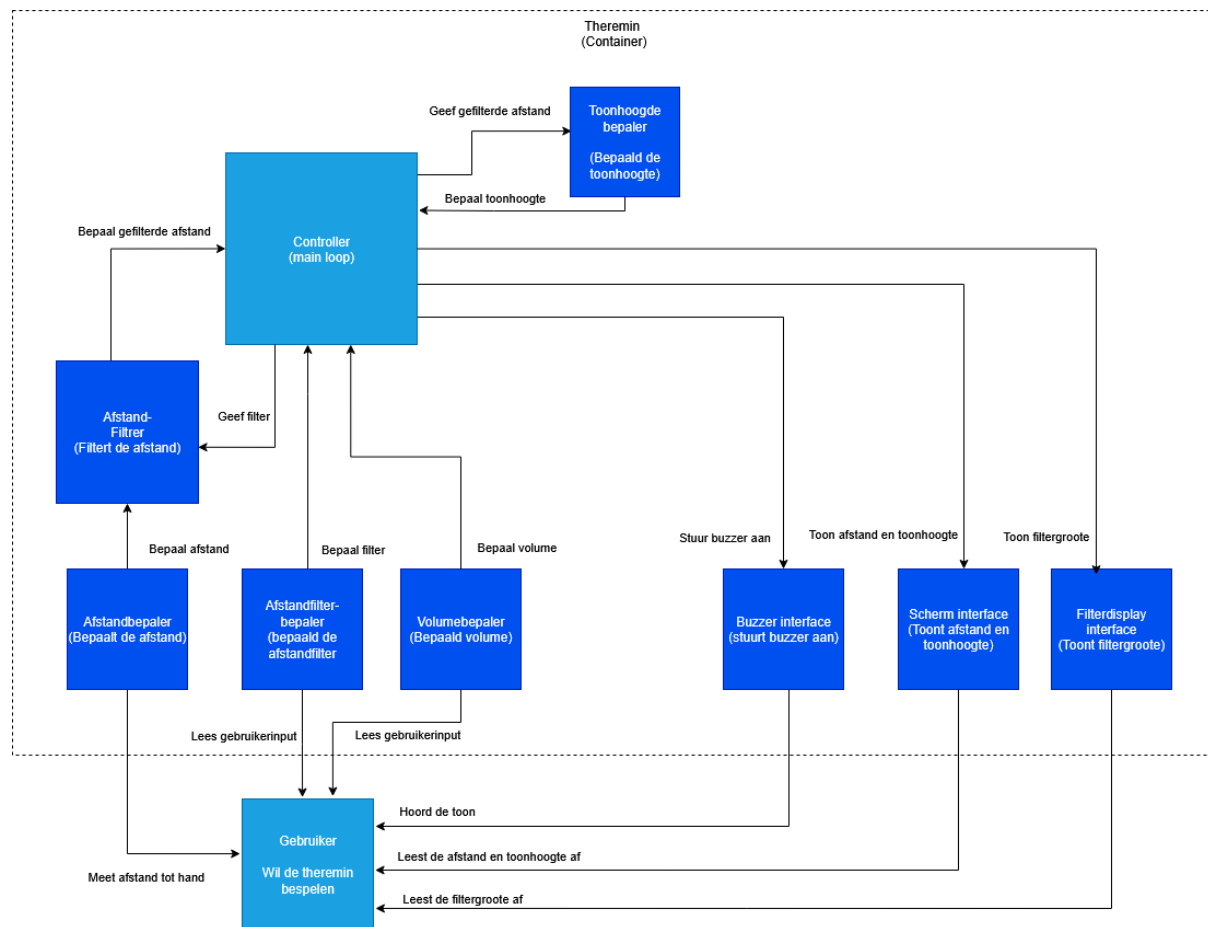
Het systeem heeft slechts 1 container. Er is geen verdeling tussen meerdere containers. Het diagram is daardoor vergelijkbaar met de context.



3 Componenten

De theremin bestaat uit een aantal verschillende componenten. Ten eerst is er een volume regelaar waarmee de gebruiker het volume kan bepalen (US.3). Ten tweede is er een sensorcomponent waarmee afstand kan worden bepaald (US.1). Met behulp van de afstand kan de toonhoogte van de theremin worden aangepast. Omdat het signaal voor de theremin zo stabiel mogelijk moet zijn, wordt het afstandssignaal gefilterd (UC.2). Dat gebeurt door het filter component. De grootte van het gebruikte filter kan door de muzikant worden aangepast (UC.6) door middel van twee knoppen: één om het filter te vergroten en één om het filter te verkleinen. Aan de hand van al deze input signalen zal de controller de output componenten gaan aansturen. Zodra er een nieuwe afstandsmeting binnen is, zal de controller de

buzzer interface gaan aansturen zodat er een toon wordt geproduceerd (UC.4). De gespeelde frequentie en afstand zal naar het schermcomponent (UC.5) worden gestuurd, zodat de gebruiker deze kan aflezen. Tevens zal de gebruikte grootte van het filter worden getoond op een 7-segmenten display (UC.7).



3.1 Componenten

De meeste componenten zijn hardware/software componenten. Het filter is puur een software component en bevat het filter algoritme voor het filteren van de gemeten afstand, vanwege variaties (ruis) in de metingen.

Omdat we voor dit project geen gebruik willen maken van callbacks is ervoor gekozen om de Controller de controle te laten houden over het hele proces. Dat betekent dat de controller het centrale punt is in het hele systeem. De controller heeft een main-loop, die de verschillende componenten aanstuurt en uitleest.

Sommige onderdelen zouden versneld kunnen worden door de interactie niet via de Controller te laten lopen, maar door rechtstreeks de componenten met elkaar te laten interacteren via een callback functie (zoals bijvoorbeeld de volume uitlezing en aansturing van de buzzer, die direct via het zetten van de PWM waarde in een register zou kunnen worden gedaan).

In het activity diagram hiernaast is de globale afloop te zien. Het risico van dit model is dat als een component langer tijd nodig heeft, de overige componenten mogelijk niet snel genoeg reageren. Als dat een probleem wordt, moet de code meer verweven worden buiten de controller om.

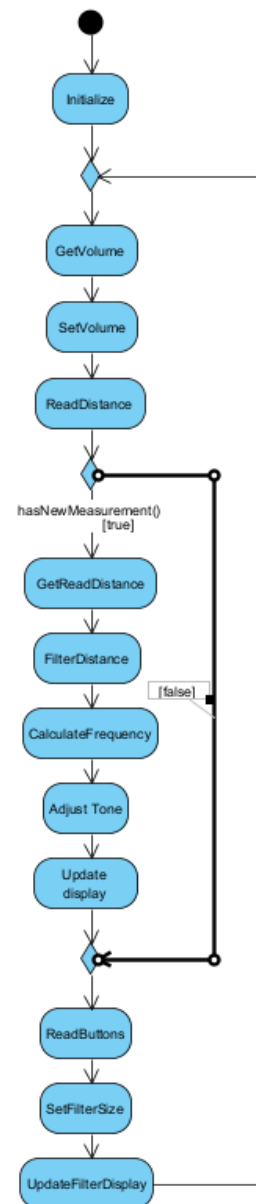
3.2 Hardware overzicht

Voor dit project wordt de hardware aangesloten zoals te vinden is in

Pin nr	Pin naam	I/O	Verbonden met	Bijzonderheden
1	PD3	Output	Buzzer	
2	PD4	Input	Knop voor filter	Interne pull-up
3	PD5	Input	Knop voor filter	Interne pull-up
4	PC0	Input	Potmeter	
5	PC4	Input	I2C-bus	Interne pull-up
6	PC5	Input	I2C-bus	Interne pull-up
7	PB0	Input	Echo ping sensor	
8	PB1	Output	Trigger ping sensor	

hoofdstukken van die componenten.

Pin nr	Pin naam	I/O	Verbonden met	Bijzonderheden
1	PD3	Output	Buzzer	
2	PD4	Input	Knop voor filter	Interne pull-up
3	PD5	Input	Knop voor filter	Interne pull-up
4	PC0	Input	Potmeter	



. De details van
Figuur 1 main afloop
 de keuzes
 hiervoor is te
 vinden in de

5	PC4	Input	I2C-bus	Interne pull-up
6	PC5	Input	I2C-bus	Interne pull-up
7	PB0	Input	Echo ping sensor	
8	PB1	Output	Trigger ping sensor	

Tabel 1 Aansluittabel

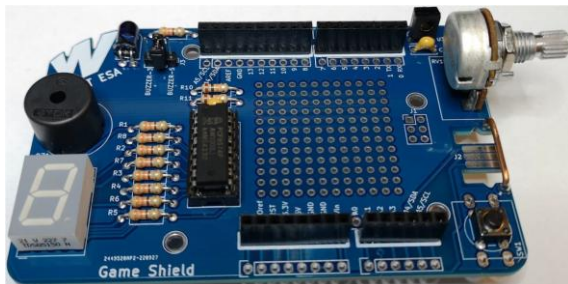
4 Controller

Voor de controller is gekozen om de Arduino Uno te gebruiken met daarop de ATmega328P microcontroller. Deze is gekozen vanwege de formfactor waarbij er eventueel makkelijk shields gemaakt kunnen worden die eenvoudig te monteren zijn. Daarnaast is dit een eenvoudige microcontroller, maar is het toch voldoende voor de doeleinden waarvoor het gebruikt gaat worden.

De controller wordt tevens uitgebreid met een gameshield. Dit gameshield kan als shield worden gemonteerd op de Arduino UNO en bevat een aantal hardware componenten die gebruikt zullen worden bij de implementatie van verschillende componenten.



Figuur 2 Arduino UNO



Figuur 3 Gameshield

4.1 Main

Zoals al is aangegeven bij het componenten diagram, zal de main-loop de verschillende onderdelen inlezen en aansturen. Sommige componenten, zoals de buzzer, maken gebruik van timers en/of interrupts. De configuratie en afhandeling van die timers of interrupts wordt in het betreffende component opgelost. Er is gekozen voor de main-loop als controle, omdat het voor deze opdracht te complex gaat worden om call-backs te gebruiken. Het kan zijn dat de code voor de werking geoptimaliseerd kan worden. Het is dan wel minder gestructureerd dan hoe het nu is opgezet, waardoor het moeilijker onderhoudbaar wordt, omdat de componenten meer afhankelijk van elkaar worden. In de main-loop wordt eerst alle initialisatie gedaan en vervolgens worden de componenten gelezen/aangestuurd.

5 Volume regelaar

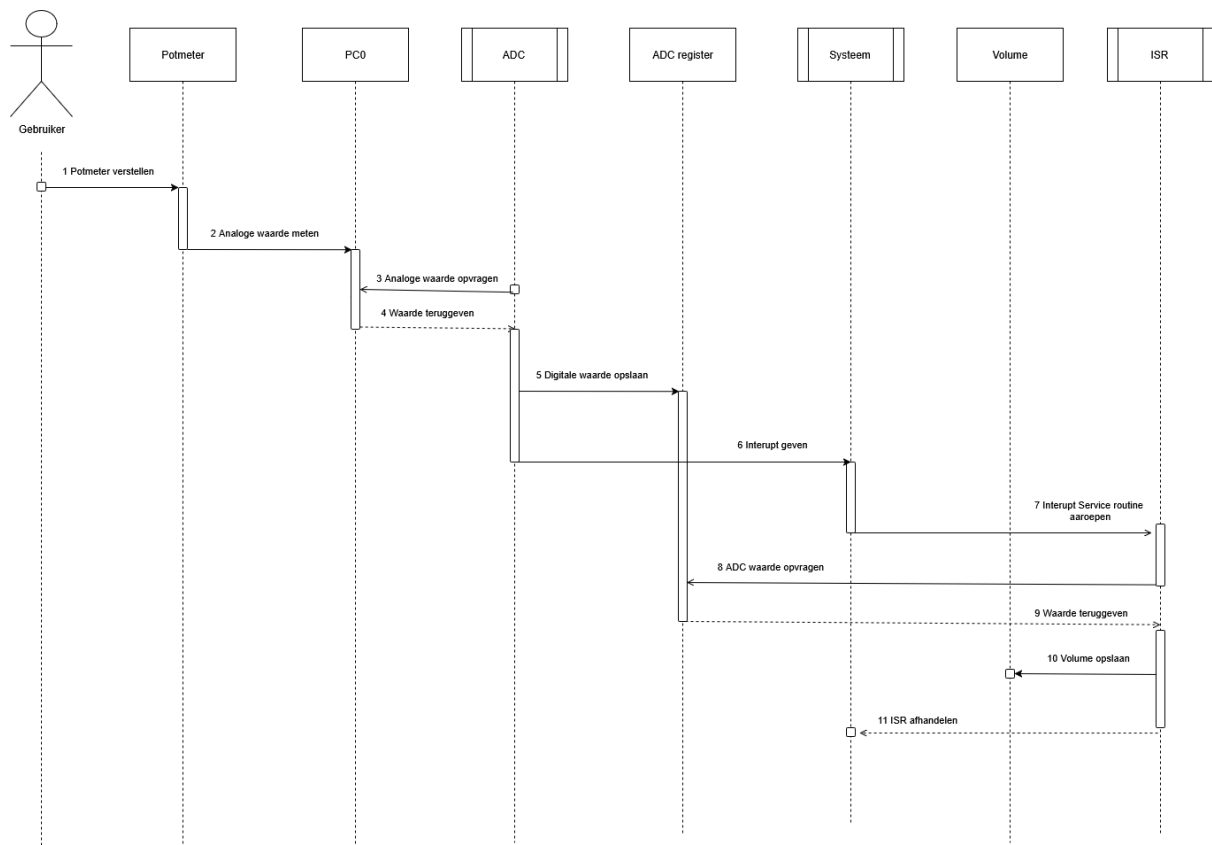
Voor het regelen van het volume van de theremin wordt een potmeter gebruikt. Deze potmeter is beschikbaar op het gebruikte gameshield. De waarde van de potmeter kan worden uitgelezen op poort PC0 van de ATmega328P.



Pin nr	Pin naam	I/O	Verbonden met
4	PC0	Input	Potmeter

Tabel 2 Aansluittabel potmeter

Figuur 4 Potmeter



Figuur 5 Sequence diagram volume bepaling

5.1 ADMUX instellingen

De gebruikte ADC heeft 10 bits, maar er worden 8 bits gebruikt voor de conversie, omdat de 2 LSB meer door ruis wordt veroorzaakt en dat voor het resultaat weinig uitmaakt. Daarvoor is het ADLAR-bit op 1 gezet in het ADMUX-register. Via het ADCH register komen de most significant 8 bits beschikbaar nadat de conversie is voltooid.

Voor de referentie voltage is gekozen voor AV_{CC} met externe condensator op de AREF pin (de condensator is geplaatst op Arduino UNO). Het analoge channel is gezet op ADC0 (=PC0). Zie hoofdstuk 24.9 in de ATmega328P datasheet voor meer informatie.

5.2 ADCSRA instellingen

De ADC moet bij het starten van de Arduino meteen gaan beginnen met het meten van de analoge waarde van de potmeter. Daarna moet er continue gemeten worden op een zo snel mogelijke manier zodat het instrument responsief is (NFE.3). Om dat te doen wordt het ADEN-bit op 1 gezet om de ADC aan te zetten en ook het ADSC-bit wordt op 1 gezet, zodat deze ook meteen automatisch zal starten met de eerste conversie. In de ADCSRB moet de ADC in freerunning mode staan, dus na de eerste start zal de ADC continue conversies doen. Omdat we een interrupt willen wanneer een conversie klaar is, is ook ADIE op 1 gezet.

Omdat er gekozen is om continue te meten en het niet telkens vanuit de main te

Pin nr	Pin naam	I/O	Verbonden met	Bijzonderheden
8	PB0	Input	Echo van ping sensor	
9	PB1	Output	Trigger van ping sensor	

starten, zullen er interrupts komen wanneer de conversie klaar is. Er is voor gekozen om de prescaler van de ADC op 128 te zetten, om de hoeveelheid interrupts tot een minimum te beperken. Bij een klokfrequentie van 16MHz en een prescaler van 128 en elke 13.5 cycles een meting, betekent het dat er maximaal $16 * 10^6 / 128 / 13.5 = 9259$ interrupts per seconde. Mocht dit te veel zijn, dan kan er voor worden gekozen om via de main een conversie te starten en het resultaat te lezen. Om de ADC automatisch te laten starten, moet Auto Trigger Enable bit (ADATE) worden aangezet.

5.3 ADCSRB instellingen

Deze is ingesteld op de freerunning mode, zodat er continue een meting wordt uitgevoerd.

6 Frequency

Voor de afstandsmeting is gekozen voor een pingsensor met 4 pins. Dat betekent dat de triggerpin en echo pin apart worden aangesloten. De triggerpin wordt voor korte tijd hoog gezet (10 μ s). Daarna geeft de pingsensor een puls, waarna er gewacht wordt op response. De duur van het hoog zijn van de echo-puls bepaalt de tijd en daarmee de afgelegde afstand. De formule voor het omrekenen is:

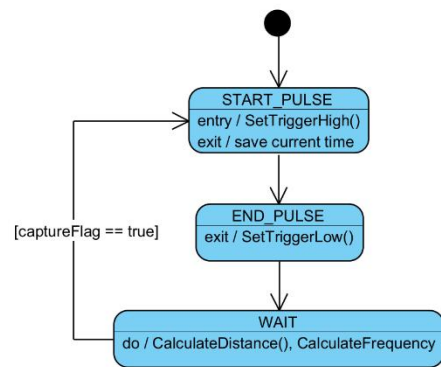


Figuur 6 4-pins HC-SR04

Afstand (cm) = tijd (μ s) / 58 (zie ook de datasheets van de pingsensor).

Er wordt een state machine gebruikt voor het versturen van de puls en het wachten op een reactie, omdat dit twee verschillende toestanden zijn waarin het component zich bevindt.

Het begint bij “START_PULSE”, waarbij het triggerbit hoog gezet wordt en de huidige tijd wordt bewaard. Daarna gaat de state machine naar de “END_PULSE” die na 10µs het triggerbit weer laag maakt. Daarna gaat de statemachine naar de “WAIT” state. Hier wordt gecontroleerd of de meting klaar is. Als dat het geval is, zal de afstand tot de sensor worden bepaald, volgens de berekening.



Figuur 7 State machine

De frequentie wordt berekend met de formule:

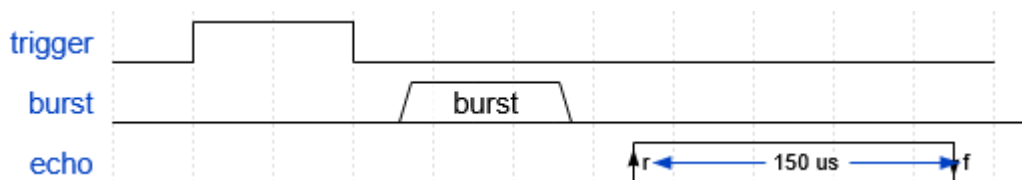
$$f = f_{max} - \frac{(f_{max} - f_{min}) * distance(cm)}{DISTANCE_{max}}$$

De minimum frequentie is ingesteld op 230Hz en het maximum op 1400Hz, zoals is aangegeven in de requirements (NFE.1).

De statemachine moet wel telkens worden aangeroepen om te functioneren.

6.1 Tijdmeting

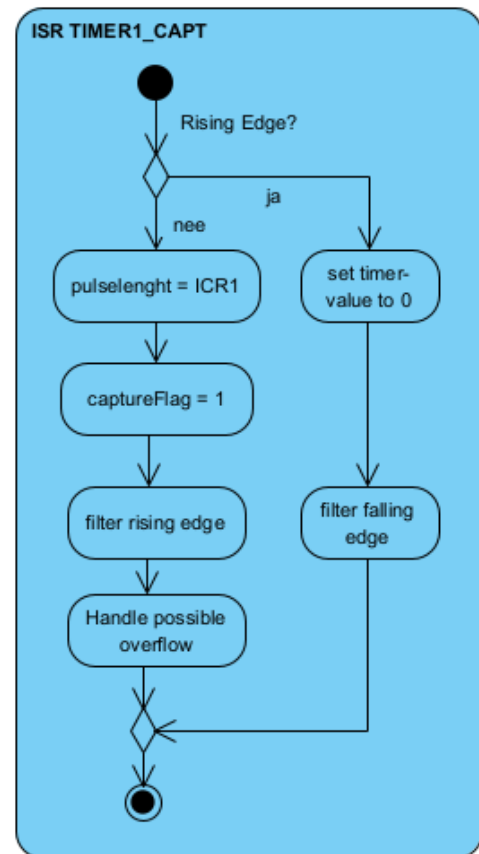
Wanneer de ping sensor een trigger krijgt, zal daarna een burst worden gegeven van 8 pulsen. Meteen daarna gaat de echo pin naar hoog. Pas als de sensor ook weer een burst ontvangt zal de echo pin weer naar omlaag gaan. De tijd tussen het omhoog en omlaag gaan is de tijdmeting die van belang is. Zie ook het plaatje.



Figuur 8 Timing ping sensor

In het voorbeeld heeft het 150 μ s, voordat de burst weer werd ontvangen. Om deze tijd te meten is ervoor gekozen om de rising edge ('r') en de falling edge ('f') op te vangen en de tijd te meten met timer 1. Met die timer is het mogelijk om te starten en te stoppen op een rising of falling edge. Dat kan met de capture input functionaliteit.

Timer 1 is een 16 bits timer. Deze timer wordt eerst zo ingesteld, dat er een interrupt komt op een rising edge. Zodra er een rising edge is ontvangen en de ISR wordt aangeroepen, wordt de timer zo ingesteld dat er een interrupt komt op een falling edge, de timer wordt gereset en wordt er een nieuwe meting gestart. Wanneer ook de falling edge is ontvangen, dan wordt de ISR weer aangeroepen en kan de gemeten tijd worden uitgelezen. De lengte van de puls is opgeslagen in het Input Capture Register (ICR1) van de timer. In het diagram hiernaast wordt dat verduidelijkt.



Figuur 9 ISR Capture Input

Omdat de statemachine moet stoppen met wachten en de afstand berekend kan worden, moet er een flag worden gezet, zodat de statemachine weet dat er een nieuwe tijdmeting is gedaan.

6.1.1 TCCR1A instellingen

Om het meten van het signaal voor elkaar te krijgen moet de timer in normal mode worden gezet. Dit register moet op 0x0 worden ingesteld.

6.1.2 TCCR1B instellingen

Vooralsnog wordt de Capture Noise Canceler niet gebruikt, maar die kan worden ingesteld als blijkt dat er toch iets van filtering nodig is.

De eerste keer moet het ICES1 bit op 1 worden gezet, om ervoor te zorgen dat er een interrupt komt bij de rising edge. Dit zal elke interrupt moeten worden aangepast om zo zowel de falling als de rising edge interrupts te krijgen, zoals hierboven is beschreven.

We maken geen waveform, deze bits kunnen op 0 blijven staan.

In de eisen staat dat er een afstand van 65cm gemeten moet worden (NFE.2). Geluid gaat met een snelheid van ongeveer 34300 cm/s. Het geluid gaat van de ping sensor eerst naar een object, en wordt vervolgens teruggekaatst. Het geluid moet dus maximaal in totaal 130cm afleggen. 130cm wordt met een snelheid van 34300 cm/s afgelegd in 3.8 ms. De timer kan maximaal tot 65535 tellen. Met een frequentie van 16.000.000Mhz, kan de timer een tijd tellen van $65536 \cdot (N/16.000.000)$ s voordat

er een overflow is en waarbij N de prescaler is. Door een prescaler van 8 te nemen kan er tot tijd van 33ms worden geteld, waarmee ruimschoots tot 0.38 kan worden geteld wat benodigd is voor de afstand van 130cm. De 33ms kan uiteindelijk ook als time-out worden gebruikt, waardoor een nieuwe meting kan worden gestart na 33 ms.

6.1.3 TIMSK1instellingen

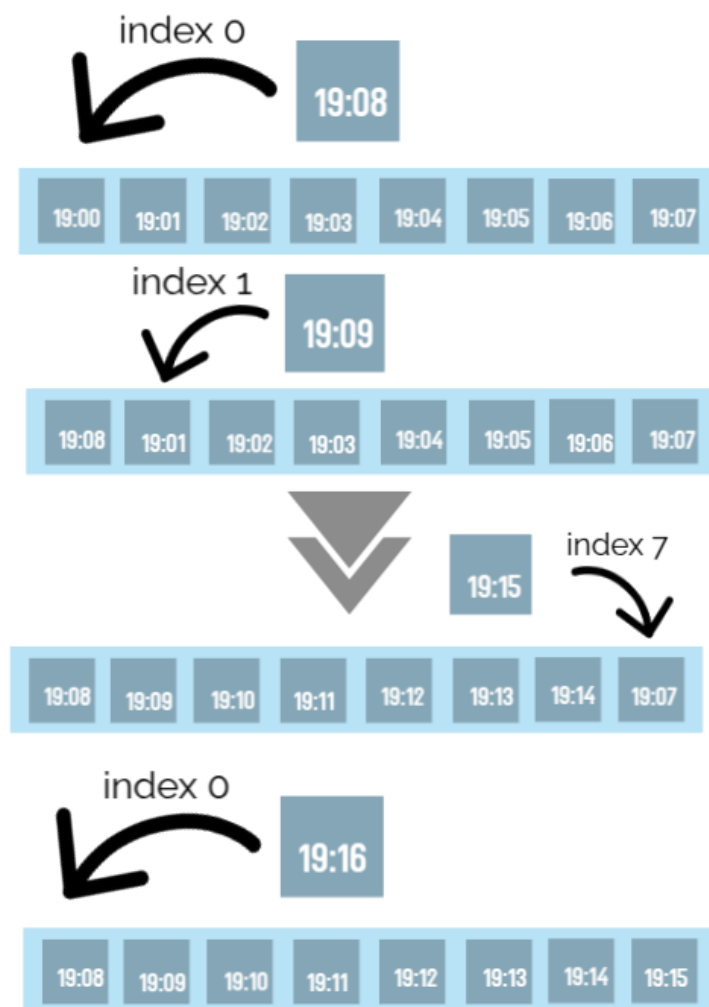
Om de Input Capture Interrupt in te schakelen, moet in dit register het ICIE1-bit op 1 worden gezet.

7 Filter

Voor het filteren is het mediaanfilter gebruikt, zoals in het functioneel ontwerp staat (FE.5).

Bij het mediaanfilter worden waarden in een lijst bewaard, gesorteerd en vervolgens wordt het middelste getal genomen als gefilterde waarde. Het midden van de array wordt bepaald door $(\text{arraylength} / 2) + 1$ te nemen. Dit werkt voor zowel een even als een oneven aantal elementen. Eventueel kan het nog aangepast worden door het gemiddelde van de twee elementen te berekenen, maar daar is nu niet voor gekozen.

Elke meting wordt in de array op een bepaalde index opgeslagen. Deze index verhoogt de hele tijd en begint na de maximale waarde weer bij 0. De oudste waarde (op de index die het langste niet veranderd is) wordt dus constant vervangen door de nieuwste waarde.



Figuur 10 Voorbeeld opslaan meest recente waarden

Omdat elke waarde in de array zijn index moet behouden zodat we weten welke we moeten vervangen, moet er opgelet worden dat er elke keer een kopie van de array gebruikt wordt om mediaan te vinden. We willen de filterfunctie dus uitvoeren met een call by value i.p.v. een call by reference. Echter, omdat een array eigenlijk op zichzelf al een reference (pointer) is, kunnen we niet direct via de functie parameter een call by value aangeven. Inplaats daarvan maken we in de functie zelf een kopie van de array met memcpy om de call by value te simuleren.

7.1 Quicksort

Voor het sorteren is gekozen voor een sorteringsalgoritme wat bij de standard C library is inbegrepen, het quicksort algoritme (`qsort()`). Dit algoritme werkt voor kleine lijsten erg effectief.

Om `qsort()` te gebruiken moet alleen een `compare()` functie worden gemaakt om de sortering van de bovengenoemde datastructuur te kunnen doen. Om de arrayelementen te sorteren op value moet de `compare()` functie waardes gaan vergelijken.

Nadat de sortering gedaan is, kan de middelste waarde worden genomen. Dat is ook meteen de mediaan.

8 Filter Instellen

Het filter dat wordt gebruikt is het mediaan filter (zie hoofdstuk 7). Deze heeft een buffer aan waardes, waarvan de mediaan wordt bepaald. Als het buffer kleiner is, zal het uitgangssignaal sneller in het ingangssignaal gaan volgen, maar de filtering is dan kleiner. Wordt het buffer groter gezet, dan wordt de filtering groter en reageert het uitgangssignaal later op verandering van het ingangssignaal.



De grootte van het filter is minimaal 1 en maximaal 15 (NFE.4).

Om de juiste gewenste grootte in te kunnen stellen, worden er 2 knoppen toegevoegd waarmee de buffer verkleint of vergroot kan worden.

Pin nr	Pin naam	I/O	Verbonden met	Bijzonderheden
2	PD4	Input	Knop voor filter	Interne pull-up
3	PD5	Input	Knop voor filter	Interne pull-up

De knoppen worden door middel van interrupts ingelezen. Daarvoor moeten de interrupts worden ingesteld en de bijbehorende pins moeten worden ingesteld.

8.1 PCICR en PCMSK2

Om de interrupts in te schakelen wordt het Pin Change Interrupt Control Register ingesteld, zodat Pin Change Interrupt Enable 2 kan worden gebruikt. Deze kan

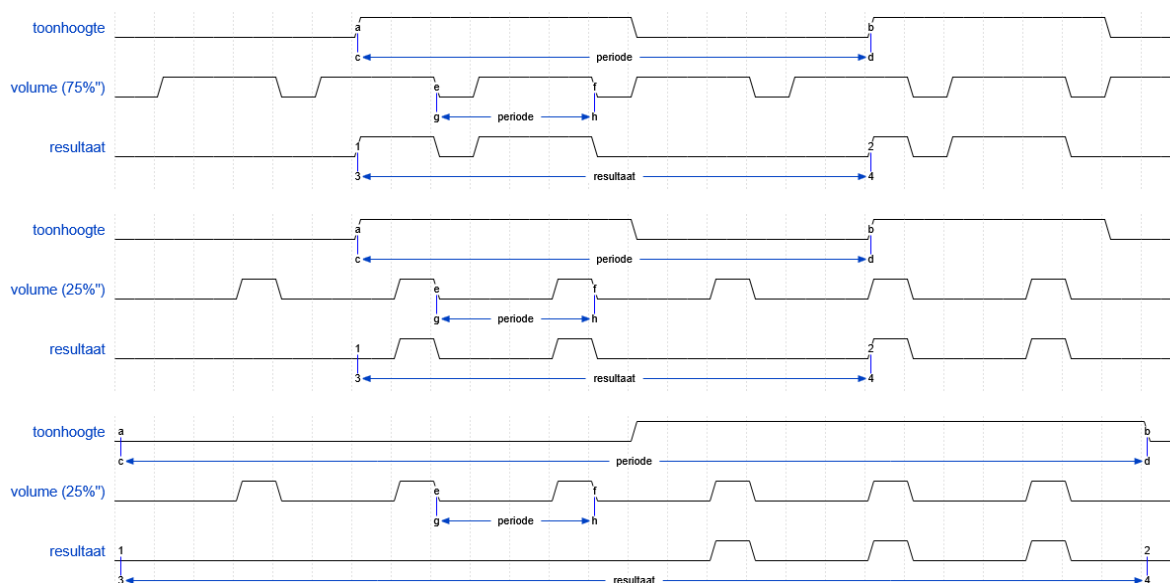
vervolgens worden ingesteld met PCINT20 en PCINT21, zodat pin 2 en 3 gebruikt kunnen worden voor de knoppen.

9 Buzzer interface

De buzzer moet zowel de toonhoogte (frequentie) alsook het volume kunnen laten horen. Er is voor gekozen om de potmeter niet tussen de output en de buzzer te zetten (analoge damping), maar om die ook digitaal in te lezen en daarmee het outputsignaal naar de buzzer te beïnvloeden, door modulatie toe te passen. Daarmee worden twee frequenties met elkaar samengevoegd.



De frequentie van de toonhoogte is een halve periode hoog en een halve periode laag (een blokgolf) en is in de hoorbare range (230Hz-1400Hz). Het toonsignaal wordt gegenereerd door de aangesloten pin aan en uit te schakelen. Om de op die manier gegenereerde toon te dempen wordt de tijd dat de periode hoog is, samengevoegd met een fast PWM-signaal gezet op maximale frequentie ($16000000\text{Hz} / 256 = 62,5\text{kHz}$). Hoe groter die pulsbreedte, hoe meer geluid er geproduceerd wordt. In de onderstaande tekeningen wordt dat verduidelijkt, al is het verschil in periode tussen beide frequenties hoger. In werkelijkheid zitten binnen een periode van timer0 tientallen PWM periodes van timer2.



In het bovenste plaatje is te zien dat de PWM op 75% staat. Zodra het bovenste signaal hoog is, zal het PWM signaal worden uitgestuurd, anders niet. Bij het tweede plaatje is hetzelfde effect te zien, maar dan bij een PWM van 25%. Het derde plaatje laat het effect zien bij een lagere frequentie (en dus een langere periode).

Pin nr	Pin naam	I/O	Verbonden met	Bijzonderheden
1	PD3	Output	Buzzer	

Er zijn 2 timers nodig, eentje voor de fast PWM en eentje voor de toonhoogte. Om het samenvoegen voor elkaar te krijgen, wordt de output van de fast PWM uitgezet, wanneer de halve periode van de frequentie laag wordt en weer aangezet wanneer de halve periode weer hoog zet. De PWM blijft gewoon doorlopen, alleen de output pin wordt dus disabled. Output PD3 is verbonden met Timer/Counter 2.

9.1 Volume (fast PWM)

Timer/counter 2 wordt gebruikt voor de fast PWM. Deze timer wordt zo ingesteld, dat deze een continu Fast PWM signaal op de outputpin zet met een zo hoog mogelijke frequentie.

9.1.1 TCCR2A instellingen

Om Fast PWM in te stellen moeten WGM21 en WGM20 op 1 worden gezet. Compare Match Output A wordt niet gebruikt en kan op 0 blijven staan. Compare Match Output B moet ingesteld worden op ophogend (non-inverting), dus moet alleen COM2B1 hoog gezet worden.

9.1.2 TCCR2B instellingen

In PWM worden FOC2 A en B niet gebruikt, die blijven 0. Waveform generation moet ook uit blijven staan. Voor de klok source, wordt de clock gebruikt, zonder prescaling. Daarmee wordt het snelste PWM-frequentie gegenereerd.

9.1.3 TIMSK2 instellingen

Alleen de interrupt voor de overflow is van belang, de overige 2 blijven uit, omdat we geen compare gebruiken.

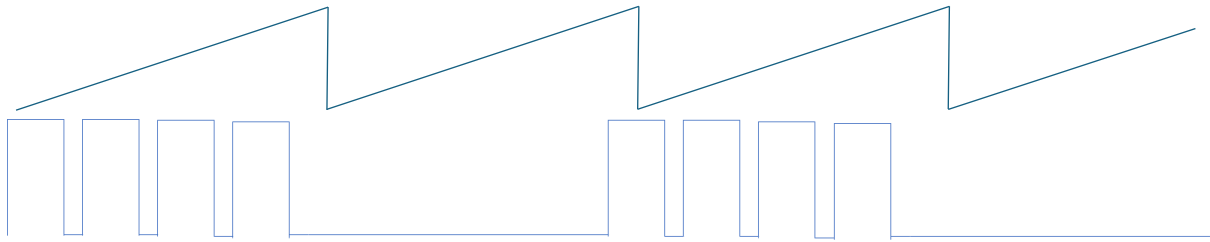
9.1.4 OCR2B instellingen

De output compare register bepaalt de duty cycle en zal bij elke aanpassing van het volume worden aangepast om zo de duty cycle aan te passen.

9.2 Frequentie

De **Timer/Counter0** wordt gebruikt om de frequentie die we willen horen op de buzzer. Zodra er een interrupt komt dat een halve periode voorbij is, moet output pin (PB3) van Timer/Counter 2 aan of uit worden gezet. We stellen de timer in op CTC mode, zodat we de frequentie kunnen bepalen door de waarde van het Output Compare Register (OCR0A) te veranderen. Hoe hoger de waarde van OCR0A, hoe langer het duurt voordat de teller weer bij 0 begint, hoe langer de periode en dus hoe lager de frequentie.

Elke keer wanneer de tellerstand van Timer/Counter0 OCR waarde bereikt, wordt er een compare interrupt uitgevoerd. De counter wordt gereset en begint weer bij 0 te tellen.



Deze interrupt wordt vervolgens gebruikt om de uitgang van Timer/Counter2 te moduleren (om te schakelen tussen de mode van de OC2B pin). We krijgen dan het volgende signaal op pin OC2B (op de buzzer):

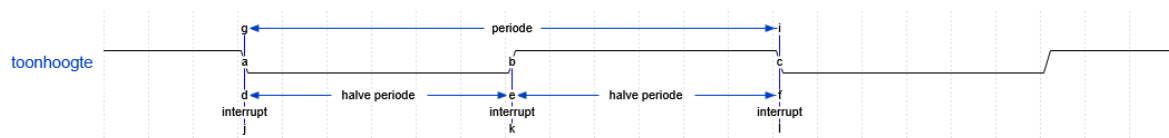
9.2.1 TCCR0A instellingen

Omdat de timer/counter van de frequentie geen outputs aanstuurt, moeten de Compare Match Output A en B op 0 worden gezet. Om in te stellen dat de counter weer bij 0 begint te tellen, moet de timer op CTC worden ingesteld. Dat moet door WGM01 op 1 zetten.

9.2.2 TCCR0B instellingen

De FOC-bits worden niet gebruikt en moeten op 0 worden ingesteld. Het WGM02 bit moet ook 0 zijn. Deze wordt in combinatie met de WGM-bits van TCCR0A gebruikt. Deze moet worden ingesteld op CTC, waardoor deze 0 moet blijven.

Voor de prescaler moet de juiste halve periode worden ingesteld, omdat er elke halve periode een interrupt gegeven moet worden die de aansturing van PB3 door de timer/counter2 aan en uit moet zetten.



De frequenties die we nodig hebben moeten volgens het functioneel ontwerp tussen de 230Hz en 1400Hz liggen. Om uit te rekenen welke prescale waarde ingesteld moet worden, moet worden uitgegaan van de 230Hz, omdat die de langste periode heeft en waar de timer het langst tijd voor nodig heeft. De Output Compare Register is 8-bits, dus kan een maximale waarde hebben van 255. Voor $230 * 2$ interrupts (2 per periode), zijn dat 460 interrupts per seconde.

De gebruikte formule is

$$f_{\text{interrupt}} = \frac{f_{\text{clk}}}{N * (1 + MAX)} \implies 460 = \frac{16000000}{N * (1 + 255)} \implies N * (1 + 255) = \frac{16000000}{460}$$

$$N = \frac{\frac{16000000}{460}}{266} \implies N = \frac{16000000}{460 * 266} = 130$$

De prescaler moet worden ingesteld op 256, omdat bij een prescaler van 64, de Output Compare Register een getal moet hebben boven de 255, wat niet past. Dat betekent dat alleen CS02 op 1 gezet moet worden.

9.2.3 TIMSK0 instellingen

Omdat Output Compare Register A gebruikt gaat worden moet alleen TOIE2 op 1 worden gezet. De overige blijven op 0.

10 LCD scherm

Voor het scherm (UC.5) is gekozen voor een 2x16 display, dat via I²C aangestuurd. De interne pull-ups worden hiervoor aangezet.

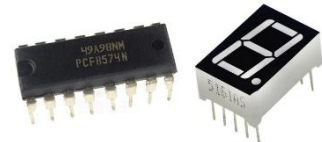
Pin nr	Pin naam	I/O	Verbonden met	Bijzonderheden
5	PC4	Input	I2C-bus	Interne pull-up
6	PC5	Input	I2C-bus	Interne pull-up

Er is gekozen voor een library die het display met eenvoudige functies kan aansturen. Deze library gebruikt de TWI-library, die ook gebruikt gaat worden voor het 7-segments display. Het scherm heeft Id 0x27.



11 7-Segmenten display

Op de gameshield is een 7-segmenten display aanwezig. Dit 7-segmenten display is aangesloten via een port expander, om pinnen te besparen van de microcontroller. De port expander is aangesloten op de I²C bus van de ATmega328P, net als het display. Ook hier wordt de TWI library gebruikt.



Pin nr	Pin naam	I/O	Verbonden met	Bijzonderheden
5	PC4	Input	I2C-bus	Interne pull-up
6	PC5	Input	I2C-bus	Interne pull-up

12 Bibliografie

ESA-team. (2025). *Functioneel ontwerp Theremin*. Zwolle: Windesheim hbo-ict.